```
#include <LiquidCrystal.h>
#include <math.h>
#include <string.h>
#include <stdlib.h>

// pin assignments
LiquidCrystal lcd(13, 12, 11, 10, 9, 8);
const byte RELAY_PIN = 7;
const byte FAULT_LED = 6;
const byte CURRENT_PIN = A0;
const byte VOLTAGE_PIN = A1;


//  Push buttons

const byte BUTTON_UP = 5;
const byte BUTTON_DOWN = 4;
const byte BUTTON_SET = 3;
const byte BUTTON_RUN = 2;
const bool BUTTON_ACTIVE_LOW = false;

const bool RELAY_ACTIVE_HIGH = true;

float overVoltageLimit = 250.0;
float underVoltageLimit = 180.0;
float overCurrentLimit = 2.0;

const float ADC_REF = 5.0;
const float ADC_MAX = 1023.0;
const float ACS_SENSITIVITY = 0.100;
const float VOLTAGE_CALIBRATION = 100.0;
const int SAMPLE_COUNT = 200;
const unsigned int SAMPLE_DELAY_US = 250;

const unsigned long MEASUREMENT_INTERVAL_MS = 10000UL;
const unsigned long DISPLAY_INTERVAL_MS = 300UL;
const unsigned long RECOVERY_DELAY_MS = 3000UL;
const unsigned long DEBOUNCE_MS = 35UL;

float voltageRMS = 220.0;
float currentRMS = 0.5;


bool manualVoltageMode = true;
bool manualCurrentMode = true;
```

```
float manualVoltage = 220.0;
float manualCurrent = 0.5;

bool relayOn = false;
bool faultLatched = false;
unsigned long safeStart = 0;
unsigned long lastMeasurement = 0;
unsigned long lastDisplay = 0;

char commandBuffer[64];
byte commandLength = 0;

enum FaultType { NORMAL, OVERVOLTAGE, UNDERVOLTAGE, OVERCURRENT, SENSOR_ERROR };
FaultType currentFault = NORMAL;

const byte buttonPins[4] = { BUTTON_UP, BUTTON_DOWN, BUTTON_SET, BUTTON_RUN };
bool buttonStable[4];
bool buttonLastReading[4];
unsigned long buttonChangedAt[4];
bool settingMode = false;
byte selectedSetting = 0; // 0=OV, 1=UV, 2=OC

bool buttonIsPressed(byte pin) {
  return BUTTON_ACTIVE_LOW ? digitalRead(pin) == LOW : digitalRead(pin) == HIGH;
}

void controlRelay(bool on) {
  relayOn = on;
  digitalWrite(RELAY_PIN, RELAY_ACTIVE_HIGH ? (on ? HIGH : LOW) : (on ? LOW :
HIGH));
}

float calculateRMS(byte pin) {
  double sum = 0.0, sumSquares = 0.0;
  for (int i = 0; i < SAMPLE_COUNT; i++) {
    float sample = analogRead(pin) * ADC_REF / ADC_MAX;
    sum += sample;
    sumSquares += (double)sample * sample;
    delayMicroseconds(SAMPLE_DELAY_US);
  }
  double mean = sum / SAMPLE_COUNT;
  double variance = sumSquares / SAMPLE_COUNT - mean * mean;
  if (variance < 0.0) variance = 0.0;
  return sqrt(variance);
}
```

```
void measureSensors() {
  if (manualVoltageMode) {
    voltageRMS = manualVoltage;
  } else {
    voltageRMS = calculateRMS(VOLTAGE_PIN) * VOLTAGE_CALIBRATION;
  }

  if (manualCurrentMode) {
    currentRMS = manualCurrent;
  } else {

    currentRMS = calculateRMS(CURRENT_PIN) / ACS_SENSITIVITY;
  }
}

bool measurementsValid() {
  return isfinite(voltageRMS) && isfinite(currentRMS) &&
         voltageRMS >= 0.0 && currentRMS >= 0.0;
}

bool measurementsSafe() {
  return measurementsValid() && voltageRMS >= underVoltageLimit &&
         voltageRMS <= overVoltageLimit && currentRMS <= overCurrentLimit;
}

void updateProtection() {
  if (!measurementsValid()) currentFault = SENSOR_ERROR;
  else if (voltageRMS > overVoltageLimit) currentFault = OVERVOLTAGE;
  else if (voltageRMS < underVoltageLimit) currentFault = UNDERVOLTAGE;
  else if (currentRMS > overCurrentLimit) currentFault = OVERCURRENT;
  else currentFault = NORMAL;

  if (currentFault != NORMAL) {
    faultLatched = true;
    safeStart = 0;
    controlRelay(false);
    digitalWrite(FAULT_LED, HIGH);
    return;
  }

  if (faultLatched) {
    controlRelay(false);
    digitalWrite(FAULT_LED, HIGH);
    return;
```

```
  }

  digitalWrite(FAULT_LED, LOW);
  if (!relayOn) {
    if (safeStart == 0) safeStart = millis();
    if (millis() - safeStart >= RECOVERY_DELAY_MS) controlRelay(true);
  }
}

const __FlashStringHelper *getFaultMessage() {
  switch (currentFault) {
    case OVERVOLTAGE: return F("OVERVOLTAGE");
    case UNDERVOLTAGE: return F("UNDERVOLTAGE");
    case OVERCURRENT: return F("OVERCURRENT");
    case SENSOR_ERROR: return F("SENSOR ERROR");
    default: return F("NORMAL CONDITION");
  }
}

void printStatus() {
  Serial.println();
  Serial.println(F("===== PROTECTOR STATUS ====="));
  Serial.print(F("VOLTAGE MODE: "));
  Serial.println(manualVoltageMode ? F("MANUAL") : F("SENSOR"));
  Serial.print(F("CURRENT MODE: "));
  Serial.println(manualCurrentMode ? F("MANUAL") : F("SENSOR"));
  Serial.print(F("CONDITION: ")); Serial.println(getFaultMessage());
  Serial.print(F("VOLTAGE: ")); Serial.print(voltageRMS, 1); Serial.println(F("
V"));
  Serial.print(F("CURRENT: ")); Serial.print(currentRMS, 2); Serial.println(F("
A"));
  Serial.print(F("RELAY: ")); Serial.println(relayOn ? F("ON") : F("OFF"));
  Serial.print(F("FAULT LATCHED: ")); Serial.println(faultLatched ? F("YES") :
F("NO"));
  Serial.print(F("OV LIMIT: ")); Serial.print(overVoltageLimit, 1);
Serial.println(F(" V"));
  Serial.print(F("UV LIMIT: ")); Serial.print(underVoltageLimit, 1);
Serial.println(F(" V"));
  Serial.print(F("OC LIMIT: ")); Serial.print(overCurrentLimit, 2);
Serial.println(F(" A"));
  Serial.println(F("============================"));
}

void printHelp() {
  Serial.println();
```

```
  Serial.println(F("VOLTAGE PROTECTOR COMMANDS"));
  Serial.println(F("HELP          - Show commands"));
  Serial.println(F("STATUS        - Show readings"));
  Serial.println(F("VOLTAGE 220   - Set simulated voltage"));
  Serial.println(F("CURRENT 1.5   - Set simulated current"));
  Serial.println(F("AUTO V        - Read voltage sensor A1"));
  Serial.println(F("AUTO I        - Read current sensor A0"));
  Serial.println(F("AUTO          - Read both sensors"));
  Serial.println(F("SET OV 250    - Set overvoltage threshold"));
  Serial.println(F("SET UV 180    - Set undervoltage threshold"));
  Serial.println(F("SET OC 2.0    - Set overcurrent threshold"));
  Serial.println(F("RESET         - Reset fault when safe"));
  Serial.println(F("Buttons: UP/DOWN browse limits; SET edit; RUN exit/reset"));
}

bool parseNumber(const char *s, float &result) {
  if (s == NULL || *s == '\0') return false;
  bool digitFound = false, dotFound = false, signAllowed = true;
  for (const char *p = s; *p != '\0'; p++) {
    if (*p >= '0' && *p <= '9') {
      digitFound = true; signAllowed = false;
    } else if (*p == '.' && !dotFound) {
      dotFound = true; signAllowed = false;
    } else if (*p == '-' && signAllowed) {
      signAllowed = false;
    } else {
      return false;
    }
  }
  if (!digitFound) return false;
  result = (float)atof(s);
  return isfinite(result);
}

void resetFaultCommand() {
  if (manualVoltageMode) voltageRMS = manualVoltage;
  if (manualCurrentMode) currentRMS = manualCurrent;

  if (measurementsSafe()) {
    faultLatched = false;
    safeStart = millis();
    controlRelay(false);
    digitalWrite(FAULT_LED, LOW);
    updateProtection();
    Serial.println(F("RESET ACCEPTED. Recovery delay started."));
```

```
  } else {
    Serial.println(F("RESET DENIED: readings are unsafe."));
  }
}

void processCommand(char *cmd) {
  if (strcmp(cmd, "HELP") == 0) { printHelp(); return; }
  if (strcmp(cmd, "STATUS") == 0) { printStatus(); return; }
  if (strcmp(cmd, "RESET") == 0) { resetFaultCommand(); return; }

  if (strncmp(cmd, "VOLTAGE ", 8) == 0) {
    float value;
    if (!parseNumber(cmd + 8, value) || value < 0.0 || value > 300.0) {
      Serial.println(F("Use VOLTAGE 220 (range 0-300 V)"));
      return;
    }
    manualVoltage = value;
    manualVoltageMode = true;
    voltageRMS = value;
    updateProtection();
    Serial.print(F("SIMULATED VOLTAGE SET TO "));
    Serial.print(value, 1); Serial.println(F(" V"));
    printStatus();
    return;
  }

  if (strncmp(cmd, "CURRENT ", 8) == 0) {
    float value;
    if (!parseNumber(cmd + 8, value) || value < 0.0 || value > 30.0) {
      Serial.println(F("Use CURRENT 1.5 (range 0-30 A)"));
      return;
    }
    manualCurrent = value;
    manualCurrentMode = true;
    currentRMS = value;
    updateProtection();
    Serial.print(F("SIMULATED CURRENT SET TO "));
    Serial.print(value, 2); Serial.println(F(" A"));
    printStatus();
    return;
  }

  if (strcmp(cmd, "AUTO") == 0) {
    manualVoltageMode = false;
    manualCurrentMode = false;
```

```
  Serial.println(F("Both analog sensors enabled."));
  return;
}
if (strcmp(cmd, "AUTO V") == 0) {
  manualVoltageMode = false;
  Serial.println(F("Voltage sensor A1 enabled."));
  return;
}
if (strcmp(cmd, "AUTO I") == 0) {
  manualCurrentMode = false;
  Serial.println(F("Current sensor A0 enabled."));
  return;
}

char *savePtr = NULL;
char *first = strtok_r(cmd, " ", &savePtr);
char *parameter = strtok_r(NULL, " ", &savePtr);
char *numberText = strtok_r(NULL, " ", &savePtr);
char *extra = strtok_r(NULL, " ", &savePtr);

if (first == NULL || strcmp(first, "SET") != 0 ||
    parameter == NULL || numberText == NULL || extra != NULL) {
  Serial.println(F("Invalid command. Type HELP."));
  return;
}

float value;
if (!parseNumber(numberText, value)) {
  Serial.println(F("Invalid numeric value."));
  return;
}

if (strcmp(parameter, "OV") == 0) {
  if (value <= underVoltageLimit || value > 300.0) {
    Serial.println(F("OV must exceed UV and be <= 300."));
    return;
  }
  overVoltageLimit = value;
} else if (strcmp(parameter, "UV") == 0) {
  if (value < 0.0 || value >= overVoltageLimit) {
    Serial.println(F("UV must be >= 0 and below OV."));
    return;
  }
  underVoltageLimit = value;
} else if (strcmp(parameter, "OC") == 0) {
```

```
    if (value <= 0.0 || value > 30.0) {
      Serial.println(F("OC must be > 0 and <= 30 A."));
      return;
    }
    overCurrentLimit = value;
  } else {
    Serial.println(F("Unknown setting. Type HELP."));
    return;
  }

  Serial.println(F("SETTING UPDATED."));
  updateProtection();
  printStatus();
}

void readSerial() {
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c == '\r' || c == '\n') {
      if (commandLength > 0) {
        commandBuffer[commandLength] = '\0';
        processCommand(commandBuffer);
        commandLength = 0;
      }
    } else if (commandLength < sizeof(commandBuffer) - 1) {
      if (c >= 'a' && c <= 'z') c = c - 'a' + 'A';
      commandBuffer[commandLength++] = c;
    } else {
      commandLength = 0;
      Serial.println(F("Command too long. Try again."));
    }
  }
}


float selectedValue() {
  if (selectedSetting == 0) return overVoltageLimit;
  if (selectedSetting == 1) return underVoltageLimit;
  return overCurrentLimit;
}

void showButtonSetting() {
  lcd.clear();
  lcd.setCursor(0, 0);
  if (selectedSetting == 0) lcd.print(F("Set OverVoltage"));
```

```
  else if (selectedSetting == 1) lcd.print(F("Set UnderVoltage"));
  else lcd.print(F("Set OverCurrent"));
  lcd.setCursor(0, 1);
  lcd.print(selectedValue(), selectedSetting == 2 ? 2 : 1);
  lcd.print(selectedSetting == 2 ? F(" A") : F(" V"));
  lcd.print(settingMode ? F(" EDIT") : F(" VIEW"));
}

void changeSetting(float direction) {
  if (selectedSetting == 0) {
    float v = overVoltageLimit + direction;
    if (v > underVoltageLimit && v <= 300.0) overVoltageLimit = v;
  } else if (selectedSetting == 1) {
    float v = underVoltageLimit + direction;
    if (v >= 0.0 && v < overVoltageLimit) underVoltageLimit = v;
  } else {
    float v = overCurrentLimit + direction * 0.1;
    if (v > 0.0 && v <= 30.0) overCurrentLimit = v;
  }
  updateProtection();
  showButtonSetting();
  printStatus();
}

void handleButton(byte index) {
  if (index == 0) { // UP
    if (settingMode) changeSetting(1.0);
    else {
      selectedSetting = (selectedSetting + 1) % 3;
      showButtonSetting();
    }
  } else if (index == 1) { // DOWN
    if (settingMode) changeSetting(-1.0);
    else {
      selectedSetting = (selectedSetting + 2) % 3;
      showButtonSetting();
    }
  } else if (index == 2) { // SET toggles edit mode
    settingMode = !settingMode;
    showButtonSetting();
    Serial.println(settingMode ? F("Button edit mode ON") : F("Button edit mode
OFF"));
  } else if (index == 3) { // RUN exits view or resets fault
    if (faultLatched) resetFaultCommand();
    settingMode = false;
```

```
    lcd.clear();
  }
}

void scanButtons() {
  for (byte i = 0; i < 4; i++) {
    bool reading = buttonIsPressed(buttonPins[i]);
    if (reading != buttonLastReading[i]) {
      buttonLastReading[i] = reading;
      buttonChangedAt[i] = millis();
    }
    if (millis() - buttonChangedAt[i] >= DEBOUNCE_MS &&
        reading != buttonStable[i]) {
      buttonStable[i] = reading;
      if (buttonStable[i]) handleButton(i);
    }
  }
}

void updateLCD() {
  if (settingMode) return;


  lcd.setCursor(0, 0);
  lcd.print(F("V:"));
  lcd.print(voltageRMS, 1);
  lcd.print(F(" I:"));
  lcd.print(currentRMS, 2);
  lcd.print(F("A  "));


  lcd.setCursor(0, 1);
  if (!measurementsValid()) {
    lcd.print(F("SENSOR ERROR    "));
  } else if (voltageRMS > overVoltageLimit) {
    lcd.print(F("VOLTAGE: HIGH   "));
  } else if (voltageRMS < underVoltageLimit) {
    lcd.print(F("VOLTAGE: LOW    "));
  } else if (currentRMS > overCurrentLimit) {
    lcd.print(F("CURRENT: HIGH   "));
  } else if (faultLatched) {
    lcd.print(F("RESET REQUIRED  "));
  } else if (relayOn) {
    lcd.print(F("V/I OK: LOAD ON "));
  } else {
```

```
    lcd.print(F("V/I OK: WAITING "));
  }
}

void setup() {
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(FAULT_LED, OUTPUT);

  for (byte i = 0; i < 4; i++) {
    pinMode(buttonPins[i], BUTTON_ACTIVE_LOW ? INPUT_PULLUP : INPUT);
    buttonStable[i] = buttonIsPressed(buttonPins[i]);
    buttonLastReading[i] = buttonStable[i];
    buttonChangedAt[i] = millis();
  }

  controlRelay(false);
  digitalWrite(FAULT_LED, HIGH);
  Serial.begin(9600);
  lcd.begin(16, 2);
  lcd.print(F("Voltage Protect"));
  lcd.setCursor(0, 1);
  lcd.print(F("Starting..."));
  delay(1000);

  manualVoltageMode = true;
  manualCurrentMode = true;
  manualVoltage = 220.0;
  manualCurrent = 0.5;
  voltageRMS = manualVoltage;
  currentRMS = manualCurrent;

  updateProtection();
  Serial.println();
  Serial.println(F("Voltage Protector Ready"));
  printHelp();
  printStatus();
  lcd.clear();

  lastMeasurement = millis();
  lastDisplay = millis();
}

void loop() {
  readSerial();
  scanButtons();
```

```
  if (millis() - lastMeasurement >= MEASUREMENT_INTERVAL_MS) {
    lastMeasurement = millis();
    measureSensors();
    updateProtection();
  }

  if (millis() - lastDisplay >= DISPLAY_INTERVAL_MS) {
    lastDisplay = millis();
    updateLCD();
  }
}
```